

Multi-Agent AI Development Workflow（完整项目代码）

项目结构

```
multi-agent-dev-workflow/  
├── app/  
│   ├── main.py  
│   ├── config.py  
│   ├── agents/  
│   ├── services/  
│   ├── workflows/  
│   └── utils/  
├── requirements.txt  
├── .env  
└── README.md
```

requirements.txt

```
fastapi  
uvicorn  
openai  
python-dotenv  
gitpython  
tiktoken  
pydantic
```

.env

```
OPENAI_API_KEY=your_api_key  
MODEL_NAME=gpt-4.1
```

config.py

```
from dotenv import load_dotenv  
import os  
  
load_dotenv()  
  
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")  
MODEL_NAME = os.getenv("MODEL_NAME", "gpt-4.1")
```

app/services/llm_service.py

```
from openai import OpenAI  
from app.config import OPENAI_API_KEY, MODEL_NAME  
  
client = OpenAI(api_key=OPENAI_API_KEY)  
  
def ask_llm(system_prompt, user_prompt):  
    response = client.chat.completions.create(  
        model=MODEL_NAME,
```

```

        messages=[
            {
                "role": "system",
                "content": system_prompt
            },
            {
                "role": "user",
                "content": user_prompt
            }
        ],
        temperature=0.3
    )

    return response.choices[0].message.content

```

app/services/repo_service.py

```

import os

def read_repo(repo_path):
    repo_content = ""

    for root, dirs, files in os.walk(repo_path):
        for file in files:
            if file.endswith((".py", ".js", ".ts", ".md")):
                file_path = os.path.join(root, file)

                try:
                    with open(file_path, "r", encoding="utf-8") as f:
                        content = f.read()

                    repo_content += f"\n\n### FILE: {file_path}\n"
                    repo_content += content[:10000]

                except Exception:
                    pass

    return repo_content

```

app/services/memory_service.py

```

conversation_memory = []

def save_memory(role, content):
    conversation_memory.append({
        "role": role,
        "content": content
    })

def get_memory():
    return conversation_memory[-20:]

```

app/workflows/workflow_engine.py

```

from app.agents.planner_agent import run_planner
from app.agents.code_agent import generate_code
from app.agents.review_agent import review_code

```

```

from app.agents.doc_agent import generate_docs
from app.agents.debug_agent import debug_code

from app.services.repo_service import read_repo

class WorkflowEngine:

    def __init__(self, repo_path):
        self.repo_path = repo_path

    def execute(self, task):

        print("\n[1] Planner Agent ■■■■...")
        plan = run_planner(task)

        print("\n[2] Repo Context ■■■■...")
        repo_context = read_repo(self.repo_path)

        print("\n[3] Code Agent ■■■■...")
        code = generate_code(task, repo_context)

        print("\n[4] Review Agent ■■■■...")
        review = review_code(code)

        print("\n[5] Doc Agent ■■■■...")
        docs = generate_docs(code)

        return {
            "plan": plan,
            "code": code,
            "review": review,
            "docs": docs
        }

```

app/main.py

```

from fastapi import FastAPI
from pydantic import BaseModel

from app.workflows.workflow_engine import WorkflowEngine

app = FastAPI()

class TaskRequest(BaseModel):
    repo_path: str
    task: str

@app.post("/run-workflow")
def run_workflow(request: TaskRequest):

    engine = WorkflowEngine(request.repo_path)

    result = engine.execute(request.task)

    return {
        "status": "success",
        "result": result
    }

```

启动项目

```
pip install -r requirements.txt
```

运行服务

```
uvicorn app.main:app --reload
```